

RAG recap: retrieve, augment, generate

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 11 of 12 · English notes

Session 11 (26 August 2026) opened in the cohort PDF folder and then recapped **retrieval-augmented generation**. Session 10 had already posed the 1,000-page PDF problem: finding one author email by stuffing the whole book into Claude wastes tokens and, past a size limit, is impossible in a single call. RAG is the pattern that fetches *only the useful slices*.

The three letters, without mysticism

1. **Retrieve** — from a corpus (PDFs, sites, databases, even video transcripts) find the few chunks that match the user question.
2. **Augment** — append those chunk texts to the prompt (question + evidence).
3. **Generate** — Claude writes the answer from that smaller, relevant context — not from 10,000 silent pages.

The LLM still generates. RAG does not replace Claude; it changes *which tokens you pay for on the way in*.

DOCS MOVE; THE PATTERN DOES NOT

Classroom vector vendors and notebook APIs change. Keep the pipeline (load → split → embed → store → search → augment → generate) and verify current Claude file/PDF behaviour on **PDF support** and citations if the exam asks about grounded answers.

Offline prep versus online query

You do not embed the library on every question. Prep happens first:

- **Load** documents into text.
- **Split / chunk** into pieces that still mean something for the business (size is a use-case choice, not a universal constant).
- **Overlap** a little (characters or sentences) when you need continuity across a cut — typical for textbooks.
- **Embed** each chunk with an *embedding model* (text in, vector out — not an LLM, which is text in, text out).
- **Store** vectors *and* the original chunk text in a vector store so search can return prose, not only numbers.

At question time: embed the **prompt**, run similarity search (cosine was the lab default; MMR and BM25 were named as other families depending on the database), take top-k chunks (two Fourier-series hits in the story instead of 175), augment, generate.

Chunking is a product decision

The demo Markdown file (a medical-research-style dataset, not an exam style guide) was split on heading patterns (##) with a regex. Lalit asked whether a Markdown parser library would do; yes — regex was

pedagogy. Other splits: character count, sentences, document structure (headings vs code fences in .md). Chunk lengths in the demo ranged from tens of characters to nearly two thousand — evidence that “one true chunk size” is a trap.

Semantic search, not string equality

You will not find the user’s sentence verbatim in a chunk. Embeddings place related meanings nearby. Cosine similarity (or the vendor’s equivalent) scores that neighbourhood. Thresholds are **vendor and method defaults** you can override; Vijay refused a single magic number for every database. Tune recall versus noise for the corpus.

Metadata versus one big pool

University example: maths, science, and AI PDFs share concepts (Fourier series in more than one subject). You can tag every chunk with file metadata and filter at query time, or dump 100+50+25 chunks into one pool and search everything. Shared vector DBs across use cases are fine; hallucination risk goes up if you retrieve the wrong faculty’s policy. Intent can also be a *tool* that chooses which collection to query — RAG is often wrapped as a tool.

How RAG sits next to tools and MCP

A retrieval function is a perfectly ordinary tool: argument is the user question (or its embedding), result is chunk text. That function can live in-process or behind MCP. Agents (chapter 03) then orchestrate “when to retrieve.” No-code platforms (Bedrock knowledge bases + S3 auto-sync, OpenSearch) implement the same three letters with consoles instead of notebooks. Claude-centric delivery in this course stayed on the **code path**.

Cost and caching

Every hit that re-embeds and re-searches costs latency and money. Lalit asked about caching: yes, at app, database, or network layer (Redis was named). That is your service design, not a Claude-only feature. Prompt caching (module 09) can still apply to the *augmented* prompt if the prefix is stable — check current cache docs.

Official documentation

- **Prompt engineering overview**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

How the augmented prompt should be written after retrieval.

- **PDF support**

<https://docs.claude.com/en/docs/build-with-claude/pdf-support>

When you send files directly versus retrieving chunks.

- **Citations**

<https://docs.claude.com/en/docs/build-with-claude/citations>

Grounded answers from provided documents.

- **Prompt caching**

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

If the RAG prefix repeats across calls.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Retrieval wrapped as a tool.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.